

Phase 5 - Redis Distributed Locks

Phase 3 vs Phase 5 implementation comparison | Backend: edunex-b | 19 September 2026

PHASE 5 CODE STATUS	JOB PROTECTED	FOCUSED TESTS	READY TO DEPLOY
PASS	4 / 4	9 / 9	YES

Recurring job audit

Job	Schedule	Before Phase 5	After Phase 5
Daily analytics	01:00 daily	Every Node process ran it.	Separate Redis lock; one executor.
Course statistics	02:00 daily	Every Node process ran it.	Separate Redis lock; one executor.
Drop-off calculation	03:00 daily	Every Node process ran it.	Separate Redis lock; one executor.
Subscription expiry	Every 30 minutes	Every Node process could mutate subscriptions and insert events.	Separate fail-safe Redis lock; one executor.

Audit result: no other recurring production scheduler was found. The server request timeout is one-shot and is not a background job.

Distributed-lock implementation

Control	Implementation
Acquire	SET key unique-token NX PX ttl. A null response means another owner holds the lock.
Ownership	crypto.randomUUID() creates a distinct token for every acquisition attempt.
Release	Atomic Lua compare-and-delete removes the key only when the stored token matches the owner.
Renewal	Token-checked Lua PEXPIRE heartbeat runs at one-third of the base TTL, keeping healthy long jobs locked without creating permanent locks.
Connection	Reuses the single Phase 3 Redis client and existing REDIS_URL/redis:// handling. No second client or library was added.
Failure policy	Production never runs unlocked. Redis/lock failure logs cron skipped because distributed lock unavailable and delays work until the next schedule.

Lock keys, TTLs, and renewal

Cron	Namespaced key	Base TTL	Heartbeat
course-stats	<CACHE_PREFIX>:production:cron:course-stats	45 min	15 min
daily-analytics	<CACHE_PREFIX>:production:cron:daily-analytics	90 min	30 min
dropoff	<CACHE_PREFIX>:production:cron:dropoff	30 min	10 min
subscription-expiry	<CACHE_PREFIX>:production:cron:subscription-expiry	15 min	5 min

The base TTL is the maximum stale-lock window after a process crash. Healthy jobs renew their own lease; a wrong or expired owner cannot renew or release a replacement owner's lock.

Files changed

File	Phase 5 change
services/distributedLock.js	New reusable acquire, renew, atomic release, execution wrapper, safe logging, and scheduler wrapper.
services/cacheService.js	Exports the existing singleton Redis client/configured state for lock reuse.
jobs/courseStatsJob.js	Protects course stats, daily analytics, and drop-off with separate lock identities and TTLs.
jobs/subscriptionTasks.js	Protects subscription expiry without changing its subscription/payment business logic.
tests/distributed-lock.test.cjs	Focused concurrency, ownership, expiry, release, failure, exception, schedule, and startup-guard tests.
package.json	Adds npm run test:cron-locks. No dependency changes.

Verification

Check	Result	Evidence
Syntax and diff validation	PASS	All changed JavaScript parsed; git diff --check clean.
Phase 5 focused suite	9 passed / 0 failed	npm run test:cron-locks
Relevant regression set	28 passed / 0 failed	Locks, Phase 3 cache, scheduler health, maintenance, and subscription access.
Complete backend suite	186 passed / 3 failed	Outside Phase 5: two annual-billing tests expect removed controller behavior; thumbnail-CDN cannot load the already-declared but locally missing S3 package.

Acceptance result

Shared Redis lock helper	PASS
Course statistics lock	PASS
Daily analytics lock	PASS
Drop-off lock	PASS
Subscription expiry lock	PASS
Unique lock ownership	PASS
Automatic lock expiry	PASS
Safe atomic release	PASS
Redis outage -> job skipped	PASS
DISABLE_BACKGROUND_JOBS preserved	PASS
Rate limiting untouched/off	PASS

Risks and deployment note

Live ElastiCache was not contacted from this workspace. The tests use a Redis-compatible mock to verify NX/PX ownership, expiry, Lua release, and concurrent execution. A Redis outage deliberately delays jobs rather than risking duplicate financial or analytics work. If connectivity fails during a running job, the heartbeat logs renewal failure and the finite lease eventually expires. No deployment occurred; AWS, PM2, frontend, rate limiting, authentication, billing logic, thumbnails, and MongoDB configuration were not changed.